Team Identifier - G
Project Name - Coder V

Project Option 2- A Code Reading Educational Tool

Milestone 4

Chauhan Avi
Desai Jainik
Pathak Ishan
Patel Karan
Shah Rahil

Project Summary- Our team's goal is to provide students and users with a visual approach to learning programming. Users can observe how code executes through an interactive node-based interface, where nodes and their connections illustrate the flow and relationships within the code. The platform also includes a quiz component and curated resources, making it a comprehensive learning experience.

Design Pattern Used - No Named Pattern - We did not follow a named design pattern. Instead, the design is inspired by the documentation-reading pattern - when developers learn a new technology, they typically refer to official documentation, most of which uses a left-side navigation layout. We adopted this convention and followed fundamental UX principles, such as placing the primary content where the eye naturally follows the page flow. Overall, we aimed for a design language that feels intuitive and engaging to navigate.

Software Components

1)High Level Components

Name	Purpose	Component pieces
Frontend (React + Vite)	Browser UI, routing, auth client, calls backend APIs	main.jsx entry; App.jsx routes; src/pages/, src/components/, src/contexts/, src/hooks/, src/api/, src/lib/, src/utils/, src/data/lessonModules.js (seed/static data); Vite/Tailwind config
Backend (Node + Express)	REST API, token verification, Supabase access, spawns Python tracer	server.js, src/app.js, src/routes/, src/controllers/, src/models/, src/middleware/, src/config/, src/services/pythonTracer.js
Python tracer	Executes user Python with sys.settrace, emits JSON steps	backend/tracer.py (stdin code ->stdout JSON)
Firebase	Identity; frontend SDK signs in; Admin SDK verifies	Client: frontend/src/lib/firebase.js,

	tokens on protected routes	AuthContext; server: backend/src/config/firebase.js, requireAuth
Supabase	PostgreSQL for users, progress, lessons, examples, quizzes, graphs	Server: backend/src/config/supabase.js + models; optional client: frontend/src/lib/supabase.js
Middleware (Express)	Runs on the request pipeline before controllers: verify identity on protected routes, reject unsafe or oversized playground code.	backend/src/middleware/auth.js - requireAuth (reads Authorization: Bearer, verifies Firebase ID token, sets req.auth). backend/src/middleware/validate.js - validateCode (checks code in body, length limit, blocked patterns) used on POST /api/trace.

Hosted Services - Vercel (frontend), Render (backend)

2)Module Class Level Components

A)Backend Routes

Name	**Purpose**	**Pieces**
healthRoutes	Liveness / config flags(Check Status of services)	GET / handler (inline)
progressRoutes	Authenticated progress	requireAuth; getProgress, markComplete, resetProgress
userRoutes	User sync	POST /sync - requireAuth, syncUser

lessonRoutes	Public lesson data	GET /, GET /:lessonId - l esson handlers
exampleRouter	Example payloads	GET /:exampleId - getExampleHandler
quizRoutes	Extra quizzes	listQuizzes, getMyAttempts (auth), getQuizDetail, submitAttempt (auth)

B) Backend Controllers

Name	**Purpose**	**Pieces**
traceController	Run code through tracer	trace
userController	Mirror Firebase user into DB	syncUser
progressController	Progress HTTP API	ensureUser (internal), getProgress, markComplete, resetProgress
lessonController	Lessons/examples HTTP API	listLessons, getLessonDetailHandler, getExampleHandler
quizController	Quiz HTTP API	listQuizzes, getQuizDetail, submitAttempt, getMyAttempts

C)Backend Models(Supabase)

Name	**Purpose**	**Pieces**
userModel	users table	ensureClient (file-private), upsertUser, deleteUser, getUserById
progressModel	user_progress	ensureClient, getProgress,

		markComplete, resetProgress
lessonModel	Lessons & examples graph	ensureClient, getAllLessons, getLessonDetail, getExampleById
quizModel	Extra quiz tables	ensureClient, getAllQuizzes, getQuizById, saveAttempt, getUserAttempts

D)Frontend Contexts

Name	**Purpose**	**Pieces**
AuthContext / AuthProvider	Firebase session	getToken, signOut, state: user, loading
useAuth	Consumer hook	--
ThemeContext / ThemeProvider	Dark/light + persistence	toggleTheme, isDark
useTheme	Consumer hook	--
LessonsContext / LessonsProvider	Lesson registry + cache	init (effect), getModuleData, refs: modulesCache, inFlight
useLessonsContext	Consumer hook	--
useLessonDetail	Single module load state	--

E)Frontend- Hooks and Utils

Name	**Purpose**	**Pieces**
useProgress	Progress in localStorage + API	loadLocal, saveLocal, rowsToProgress; hook API: markComplete, isComplete, getDifficultyProgress,

		getLessonProgress, getTotalProgress, progressLoading
lessonProgressGates	Gating rules	LESSON_QUIZ_PASS_THRESHOLD_PCT, isDifficultyAccessible, isExampleUnlocked, getSuggestedExampleId

F)Frontend - Layout

Name	**Purpose**	**Pieces**
AppLayout	Sidebar + topbar + content	Sidebar, Topbar, defaultTabs
Sidebar	Nav + mobile drawer	resize/click effects, signOut
Topbar	Tab nav + theme	SunIcon, MoonIcon
ProtectedRoute	Auth gate	spinner / Navigate
ErrorBoundary	Catches render errors (class component)	constructor, getDerivedStateFromError, componentDidCatch, handleReset, render
ScrollToTop	Scroll on navigation	useLocation effect
StepVisualizer	Animated steps + variable graph from steps	TYPE_COLOR, colorFor, labelFor, buildGraph, layoutNodes, default export StepVisualizer
GraphPreview	Static SVG graph for an example	NODE_COLORS, getNodePositions, getLinePoint, nodeColor, default export
QuizSection	Inline lesson quiz UI	letter options, submit / retry,

		onPass
SkeletonCard	Loading placeholders	Shimmer, SkeletonCard, SkeletonHero, SkeletonList

G)Frontend - Pages

Name	**Purpose**	**Pieces (typical)**
Login / Signup	Auth screens	form + Firebase + optional apiRequest sync
Dashboard	Home / onboarding	uses layout, progress
LessonsLanding / LessonDetail / LessonPractice	Lesson flow	contexts, useProgress, QuizSection, StepVisualizer, fetchExample, gates
GraphModule	Legacy “type-1” module with local graph	local GraphPreview, getNodePositions, getLinePoint, nodeColor, StepVisualizer, lessonTypeOneModule
Playground	Free-form trace	traceCode, editor, StepVisualizer-style UX
Quiz / QuizTake	Extra timed quizzes	quiz API, attempts
AIAssistant	AI help surface	--
Resources / About	Static for Improving SEO	--
NotFound	404	--

3)Methods / Functions

A)tracer.py file

Function / method	Purpose	Uses / contains
safe_repr	Truncate repr for JSON	--
get_str_locals	Snapshot frame locals as strings	safe_repr
make_action	Diff locals ->create / update action	--
trace_and_capture	Main trace loop	StdoutCapture, inner tracer, commit_step, exec, error handling
StdoutCapture.write / .flush	Capture print	appends to captured_prints
commit_step (nested)	Append step + flush prints	make_action
tracer (nested)	sys.settrace callback	call / line / return events

B)python- tracer.js

Function	Purpose
getPythonCmd	Resolve PYTHON_CMD env
runPythonTrace	spawn Python, write stdin, parse JSON, timeout

C)StepVisualizer.jsx (key helpers)

Function	Purpose
colorFor / labelFor	Style and text from action
buildGraph	Derive nodes/edges from steps slice
layoutNodes	Position graph for display

StepVisualizer (default)	UI state: playback, current step, graph rendering

D)GraphPreview.jsx (and duplicated Helpers in GraphModule.jsx)

Function	**Purpose**
getNodePositions	Circular layout
getLinePoint	Edge trim at node radius
nodeColor	Color from latest matching step
GraphPreview	SVG render

E)UserProgress.js

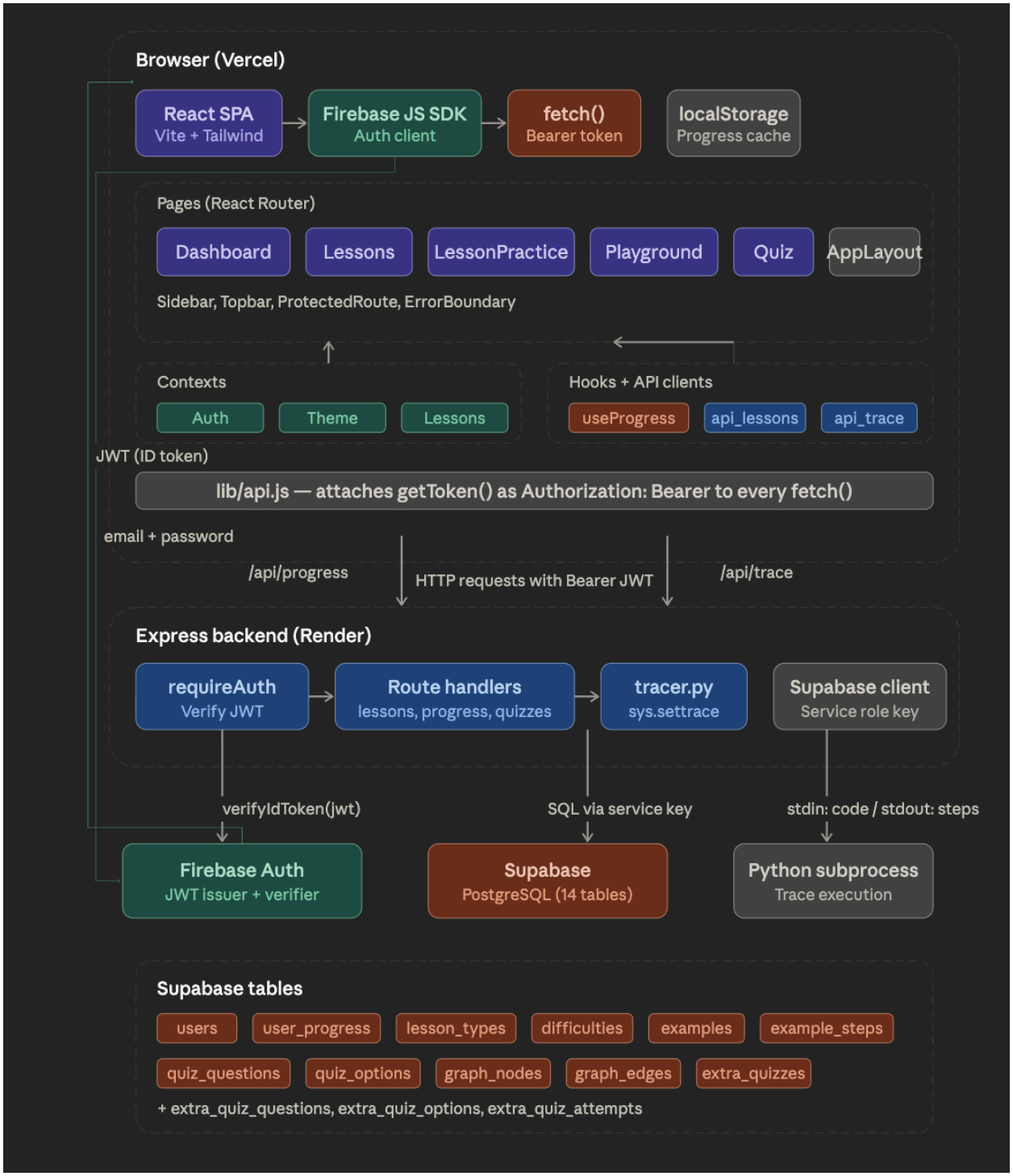
Function	**Purpose**
loadLocal / saveLocal	localStorage
rowsToProgress	Flat API rows ->nested lessonId ->difficulty ->exampleIds[]

Wiring Diagram

AI note- We drew diagram on paper(which was not very presentable), gave it to Claude Opus 4.6 to convert it into presentable diagram, ***[No AI was used for designing the Diagram]***

Legend-
Purple - Frontend UI components (React pages, SPA shell)
Teal - Authentication layer (Firebase JS SDK, AuthContext, Firebase Auth service)
Blue - Backend components (Express middleware, route handlers, API clients)
Coral - Data layer (Supabase, fetch(), useProgress hook, database tables)
Gray - Infrastructure and utilities (localStorage, lib/api.js, Python subprocess, AppLayout)

Diagram 1 - High Level System Overview


Browser (Vercel)
React SPA
Vite + Tailwind
Firebase JS SDK
Auth client
fetch()
Bearer token
localStorage
Progress cache
Pages (React Router)
Dashboard
Lessons
LessonPractice
Playground
Quiz
AppLayout
Sidebar, Topbar, ProtectedRoute, ErrorBoundary
Contexts
Auth
Theme
Lessons
Hooks + API clients
useProgress
api_lessons
api_trace
JWT (ID token)
lib/api.js — attaches getToken() as Authorization: Bearer to every fetch()
email + password
/api/progress
HTTP requests with Bearer JWT
/api/trace
Express backend (Render)
requireAuth
Verify JWT
Route handlers
lessons, progress, quizzes
tracer.py
sys.settrace
Supabase client
Service role key
verifyIdToken(jwt)
SQL via service key
stdin: code / stdout: steps
Firebase Auth
JWT issuer + verifier
Supabase
PostgreSQL (14 tables)
Python subprocess
Trace execution
Supabase tables
users
user_progress
lesson_types
difficulties
examples
example_steps
quiz_questions
quiz_options
graph_nodes
graph_edges
extra_quizzes
+ extra_quiz_questions, extra_quiz_options, extra_quiz_attempts

Persistence

Authoritative / server-side (multi-device, durable)

- User accounts - We are storing Firebase UID, email, name, optional image URL; synced when users sign up or log in ([users]).
- Lesson completion - Which examples a user/student has completed, keyed by user_id, lesson_id, difficulty, example_id ([user_progress]).
- Extra timed quizzes - The data about Quiz metadata, questions, options are also stored([extra_quizzes] –[extra_quiz_options]).
- Quiz results - The data for each user as Per-attempt score, totals, time, timestamp, ([extra_quiz_attempts]).

Client-side (Local or session storage, not on the cloud database)

- Progress cache - Nested map of completed example IDs under key coderv-progress in localStorage (JSON), hydrated from the API when signed in and updated on complete; used for resilience/offline display when backend is unavailable (frontend/src/hooks/useProgress.js).
- Theme preference - "dark" or "light" under coderv-theme in localStorage (frontend/src/contexts/ThemeContext.jsx).

Authentication tokens / session

- As we are using Google's Firebase Auth that manages persistence of the auth session in the browser (Firebase's default storage); the app uses the ID token on API calls. User identity and password handling live in Firebase (cloud), not in the PostgreSQL schema beyond the mirrored users row.

Languages Used

1. JavaScript - frontend

- Where: frontend/ (React 19, Vite, React Router, Tailwind, client-side Firebase/Supabase SDKs).
- Role: SPA UI (lessons, graph/visualization, quizzes, auth flows in the browser).
- *Why This-We found that the Node (graph/visualization) libraries in the JavaScript ecosystem are significantly more mature and widely adopted, giving us access to active community forums and a large number of open-source examples to reference.*

2. JavaScript (Node.js) - backend

- Where: backend/ - mainly server.js and related Express routes/middleware
- Role: REST API (/api/*), Firebase Admin token checks, Supabase access, orchestration; it spawns the tracer rather than reimplementing tracing in JS.
- *Why This- Four out of five developers had prior experience working with both Node.js and Supabase, making them the most practical choices for our team.*

3. Python - execution tracer

- Where: backend/tracer.py.
- Role: Runs learner Python with tracing (e.g. sys.settrace–style stepping), so step-by-step visualization comes from a Python subprocess, not from Node.
- *Why This- Python is straightforward, and since we are not performing any complex tasks that would require a compiler-heavy language, it was the natural choice. We briefly considered implementing everything purely in JSX (JavaScript with React), but concluded that the codebase would likely become less maintainable and harder to debug.*

Deployment-

We will be deploying on the web. The application is primarily designed as a desktop experience, though we will work to make it more mobile-friendly as well, however, we do not yet have a clear vision for the mobile UI. For hosting, we are using cloud services: Vercel for the frontend (we had available credits to use), Render for the backend (three out of five developers had prior experience with it), and Supabase for data storage (its free tier offers 500 MB, which is more than sufficient since our data consists only of text and JSON).

AI statement

We used AI for the Diagrams*(Not The Design)*, checking grammar mistakes and also for proof reading for the software components part.